

```
In [78]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import pylab as py
from sklearn.metrics import classification_report

from sklearn import tree
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import PredictionErrorDisplay
```

```
In [79]: npa = pd.read_csv("C:\\Users\\jdccli\\Downloads\\NPA_HLT.csv")

npa = npa.rename(columns={"NPAs_NPA": "NPA", "NPA_Lookup__Median_Sales_Price": "Median_Sales_Price"})
#Dropped columns not needed
npa = npa[["NPA", "Median_Sales_Price"]]
##Convert to numeric
npa["Median_Sales_Price"] = pd.to_numeric(npa["Median_Sales_Price"], errors="coerce")
##Drop missing values
npa = npa.dropna(subset=["Median_Sales_Price"])
##Created classification target
threshold = npa["Median_Sales_Price"].median()
npa["Affordable"] = (npa["Median_Sales_Price"] < threshold).astype(int)
npa.head()
```

```
Out[79]:
```

	NPA	Median_Sales_Price	Affordable
0	188	354750	0
1	131	215000	1
2	64	189500	1
3	326	522500	0
4	336	240000	1

```
In [80]: employment = pd.read_csv("C:\\Users\\jdccli\\Downloads\\Employment.csv")
##Removed % and converted to float
employment["Employment"] = employment["2023"].str.replace("%", "").replace("--", "No")
employment = employment[["NPA", "Employment"]]
employment.head()
```

Out[80]:

	NPA	Employment
0	2	95.6
1	3	97.4
2	4	94.3
3	5	82.8
4	6	100.0

```
In [81]: pop = pd.read_csv("C:\\Users\\jdcli\\Downloads\\Population Density.csv")
##Converted to numeric
pop["Population_Density"] = pd.to_numeric(pop["2020"], errors="coerce")
pop = pop[["NPA", "Population_Density"]]
pop.head()
```

Out[81]:

	NPA	Population_Density
0	2	5
1	3	10
2	4	4
3	5	5
4	6	4

```
In [82]: edu = pd.read_csv("C:\\Users\\jdcli\\Downloads\\Education Level - Bachelor's Degree")
edu["Bachelors"] = (edu["2023"].str.replace("%", "").replace("--", None).astype(float)
edu = edu[["NPA", "Bachelors"]]
edu.head()
```

Out[82]:

	NPA	Bachelors
0	2	35.2
1	3	85.4
2	4	89.4
3	5	2.5
4	6	21.0

```
In [83]: crime = pd.read_csv("C:\\Users\\jdcli\\Downloads\\Crime - Property.csv")
crime["Crime_Property"] = pd.to_numeric(crime["2022"], errors="coerce")
crime = crime[["NPA", "Crime_Property"]]
crime.head()
```

Out[83]:

	NPA	Crime_Property
0	2	60.0
1	3	60.0
2	4	7.0
3	5	26.0
4	6	81.0

```
In [84]: ## Merged all the dataframes together
df = employment.merge(edu, on="NPA", how="left")
df = df.merge(pop, on="NPA", how="left")
df = df.merge(crime, on="NPA", how="left")
df = df.merge(npas, on="NPA", how="left")
final = df.copy()
final.head()
```

Out[84]:

	NPA	Employment	Bachelors	Population_Density	Crime_Property	Median_Sales_Price
0	2	95.6	35.2	5	60.0	535000
1	2	95.6	35.2	5	60.0	323750
2	2	95.6	35.2	5	60.0	325000
3	2	95.6	35.2	5	60.0	375000
4	2	95.6	35.2	5	60.0	565000



```
In [85]: ##Cleaning data
final = final.dropna(subset=["Employment", "Bachelors", "Population_Density", "Crime_Property"])
##Defined features and target
X = final[["Employment", "Bachelors", "Population_Density", "Crime_Property"]]
y = final["Affordable"]

## Train and test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

##Fit Logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

##Evaluation results
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.6996402877697842

Confusion Matrix:

```
[[216  83]
```

```
[ 84 173]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.72	0.72	0.72	299
1	0.68	0.67	0.67	257
accuracy			0.70	556
macro avg	0.70	0.70	0.70	556
weighted avg	0.70	0.70	0.70	556

```
In [86]: coef_df = pd.DataFrame({"Feature": X.columns, "Coefficient": model.coef_[0]})
coef_df
```

Out[86]:

	Feature	Coefficient
--	---------	-------------

0	Employment	0.012785
1	Bachelors	-0.050445
2	Population_Density	0.038946
3	Crime_Property	-0.001462

1. Higher employment rate slightly increases the likelihood of a neighborhood being affordal as some areas with low unemployment may not necessarily be high income.
2. String negative indicating neighborhoods with higher percent of college educated adults and more likely to be expensive.
3. Higher density may reflect that denser neighborhoods tend to be more affordable.
4. Crime has a very small negative percentage, indicating it is not a strong predictor of affordability as it implies higher crime rates equal likely to be more expensive

```
In [87]: ##KNN model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("KNN Confusion Matrix:\n", confusion_matrix(y_test, knn_pred))
print("KNN Classification Report:\n", classification_report(y_test, knn_pred))
```

KNN Accuracy: 0.6888489208633094

KNN Confusion Matrix:

```
[[201  98]
 [ 75 182]]
```

KNN Classification Report:

	precision	recall	f1-score	support
0	0.73	0.67	0.70	299
1	0.65	0.71	0.68	257
accuracy			0.69	556
macro avg	0.69	0.69	0.69	556
weighted avg	0.69	0.69	0.69	556

```
In [88]: ##Decision Tree model
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
print("Decision Tree Confusion Matrix:\n", confusion_matrix(y_test, dt_pred))
print("Decision Tree Classification Report:\n", classification_report(y_test, dt_pr
```

Decision Tree Accuracy: 0.7230215827338129

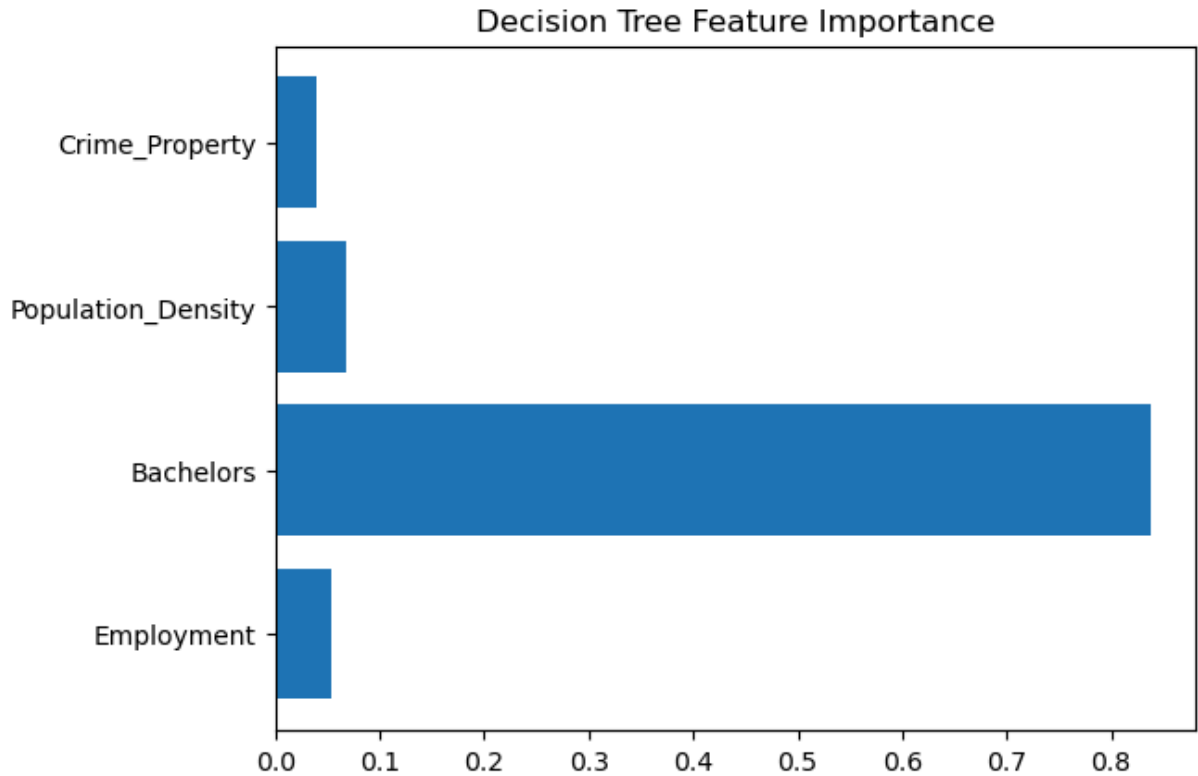
Decision Tree Confusion Matrix:

```
[[187 112]
 [ 42 215]]
```

Decision Tree Classification Report:

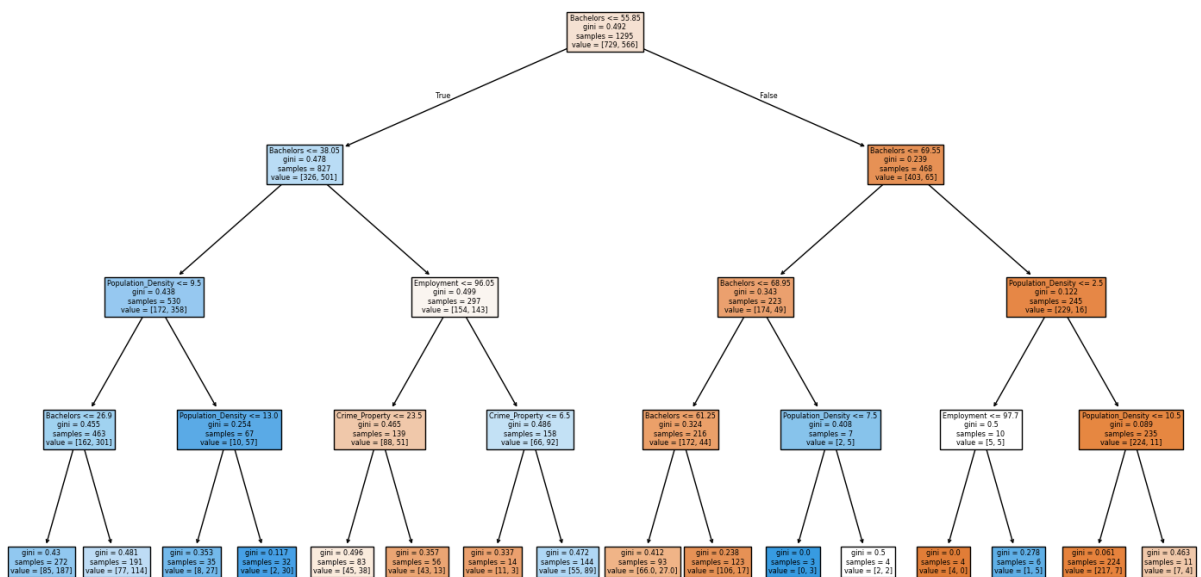
	precision	recall	f1-score	support
0	0.82	0.63	0.71	299
1	0.66	0.84	0.74	257
accuracy			0.72	556
macro avg	0.74	0.73	0.72	556
weighted avg	0.74	0.72	0.72	556

```
In [89]: ##Feature importance from decision tree
importances = dt.feature_importances_
plt.barh(X.columns, importances)
plt.title("Decision Tree Feature Importance")
plt.show()
```



Bachelors showed to have the most significant impact on price.

```
In [90]: fig = plt.figure(figsize=(25,20))
tree.plot_tree(dt, feature_names=['Employment', 'Bachelors', 'Population_Density',
fig = plt.gcf()
fig.set_size_inches(18, 10)
```



```
In [91]: ##LDA model
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
```

```
lda_pred = lda.predict(X_test)
print("LDA Accuracy:", accuracy_score(y_test, lda_pred))
print("LDA Confusion Matrix:\n", confusion_matrix(y_test, lda_pred))
print("LDA Classification Report:\n", classification_report(y_test, lda_pred))
```

LDA Accuracy: 0.7050359712230215

LDA Confusion Matrix:

```
[[216  83]
 [ 81 176]]
```

LDA Classification Report:

	precision	recall	f1-score	support
0	0.73	0.72	0.72	299
1	0.68	0.68	0.68	257
accuracy			0.71	556
macro avg	0.70	0.70	0.70	556
weighted avg	0.71	0.71	0.71	556

```
In [92]: ##QDA model
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
qda_pred = qda.predict(X_test)

print("QDA Accuracy:", accuracy_score(y_test, qda_pred))
print("QDA Confusion Matrix:\n", confusion_matrix(y_test, qda_pred))
print("QDA Classification Report:\n", classification_report(y_test, qda_pred))
```

QDA Accuracy: 0.6888489208633094

QDA Confusion Matrix:

```
[[192 107]
 [ 66 191]]
```

QDA Classification Report:

	precision	recall	f1-score	support
0	0.74	0.64	0.69	299
1	0.64	0.74	0.69	257
accuracy			0.69	556
macro avg	0.69	0.69	0.69	556
weighted avg	0.70	0.69	0.69	556

Overall, the classification models performed solidly for using neighborhood level factors to predict whether an area is affordable or expensive. All accuracy score were within the margin of 0.69 - 0.72 indicating the models performed well. The Decision Tree performed the best with 0.72.